



MUHAMMAD TAHA

22F-3277



DECEMBER 21, 2023
COAL LAB FINAL

Q1(i) DRY RUN AND WRITE REGISTERS'S VALUE

org 0x100

jmp start

start:

mov ax, 0xA193

mov bx, 0x45AF

mov cx, 0102

mov dx, 0xE693

xor ax,bx

add ax,dx

and bx,ax

sub ah,dl

mul bl

ror ax,3

div cl

mov ax, 0x4c00

int 0x21

DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: AFD

Register	Value	Register	Value	Register	Value	Register	Value	Stack	Offset	Value	Flags
AX	3C74	SI	0000	CS	19F5	IP	011E		+0	0000	7200
BX	408F	DI	0000	DS	19F5				+2	20CD	
CX	0066	BP	0000	ES	19F5	HS	19F5		+4	9FFF	OF DF IF SF ZF AF PF CF
DX	E693	SP	FFFE	SS	19F5	FS	19F5		+6	EA00	0 0 1 0 0 0 0 0

CMD >S or SI or SYM

Address	Instruction	Operation	Value
011C	F6F1	DIV	CL
011E	B8004C	MOV	AX, 4C00
0121	CD21	INT	21
0123	8B46F6	MOV	AX, [BP-0A]
0126	D1E0	SHL	AX, 1
0128	D1E0	SHL	AX, 1
012A	C55ED8	LDS	BX, [BP-28]
012D	01C3	ADD	BX, AX
012F	8B07	MOV	AX, [BX]

Address	Value	Address	Value
DS:0000	CD 20 FF 9F 00 EA F0 FE	DS:0008	AD DE 1B 05 C5 06 00 00
DS:0010	18 01 10 01 18 01 92 01	DS:0018	01 01 01 00 02 FF FF FF
DS:0020	FF FF FF FF FF FF FF FF	DS:0028	FF FF FF FF EB 19 C0 11
DS:0030	A2 01 14 00 18 00 F5 19	DS:0038	FF FF FF FF 00 00 00 00
DS:0040	05 00 00 00 00 00 00 00	DS:0048	00 00 00 00 00 00 00 00

1 Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

Q1(ii) Find the minimum element from array 69h,87h,96h,45h,75h, don't use cx register.

org 0x100

jmp start

grade: dw 69h,87h,96h,45h,75h

start:

mov dx, 5 ; length of array

dec dx

shl dx, 1 ;points to last index of array

mov ax, [grade+si]

mov si, 2

l1:

cmp ax, [grade+si]

jl skip

mov ax, [grade+si]

skip:

add si, 2

cmp si, dx

jne l1

mov ax, 0x4c00

int 0x21

DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: AFD

AX	SI	CS	IP	Stack	Flags
0045	0008	19F5	012C	+0 0000	7244
BX	DI	DS	19F5	+2 20CD	
CX	BP	ES	19F5	+4 9FFF	OF DF IF SF ZF AF PF CF
DX	SP	SS	19F5	+6 EA00	0 0 1 0 1 0 1 0

{R} reg=value

CMD >SS

Address	Disassembly	Comment
012A 75EE	JNZ	011A
012C B804C	MOV	AX, 4C00
012F CD21	INT	21
0131 8B5702	MOV	DX, [BX+02]
0134 85D2	TEST	DX, DX
0136 7504	JNZ	013C
0138 85C0	TEST	AX, AX
013A 741C	JZ	0158
013C C746DC0000	MOV	[BP-24], 0000

Address	Hex	ASCII
DS:0000	CD 20 FF 9F 00 EA F0 FE	
DS:0008	AD DE 1B 05 C5 06 00 00	
DS:0010	18 01 10 01 18 01 92 01	
DS:0018	01 01 01 00 02 FF FF FF	
DS:0020	FF FF FF FF FF FF FF FF	
DS:0028	FF FF FF FF EB 19 C0 11	
DS:0030	A2 01 14 00 18 00 F5 19	
DS:0038	FF FF FF FF 00 00 00 00	
DS:0040	05 00 00 00 00 00 00 00	
DS:0048	00 00 00 00 00 00 00 00	

Address	Hex	ASCII
DS:0103	69 00 87 00 96 00 45 00	i . g . u . E . u . . . J T F
DS:0113	8B 84 03 01 BE 02 00 3B	ï ä ; ä ï ä .
DS:0123	01 81 C6 02 00 39 D6 75	. ü . . 9 u € . L = ! i W
DS:0133	02 85 D2 75 04 85 C0 74	. à u . à t . F . . Ä ^
DS:0143	FC 83 7D 0E 00 74 09 8B	" â } . . t . i F > H ; F ÷ ~ .

1 Step 2 ProcStep 3 Retrieve 4 Help ON 5 BRK Menu 6 7 up 8 dn 9 le 10 ri



Ax has the lowest value, which is at 0109

Q2 (i) Print the string mY nAme is kHan using BIOS Service. And convert the string to all capital letters.

```
org 0x100
jmp start
string: db 'mY nAme is kHan'
clearscreen:
```

```
    push ax
    push es
    push cx
    push di
```

```
    mov ax, 0xb800
    mov es, ax
    mov di, 0
```

```
    mov cx, 2000
    mov ax, 0x0720
```

```
    rep stosw
```

```
    pop di
    pop cx
    pop es
    pop ax
    ret
```

```
convert:
```

```
    push bp
    mov bp, sp
    push si
    push cx
    push ax
```

```
    mov si, [bp+6]
    mov cx, [bp+4]
    cld
```

```
l1:
```

```
    lodsb
```

```
    cmp al, 0x20
    je skip
    cmp al, 97
    jle skip
    add byte[si], 32
```

```
skip:
```

```
    loop l1
```

```
pop ax
pop cx
pop si
pop bp
ret 4
```

start:

```
call clearscreen
```

```
mov ah, 0x13;service
mov al, 1 ;sub service
mov bh, 0 ; page
mov bl, 07 ;attribute
mov dx, 0x11;row col
mov cx, 14 ;len
```

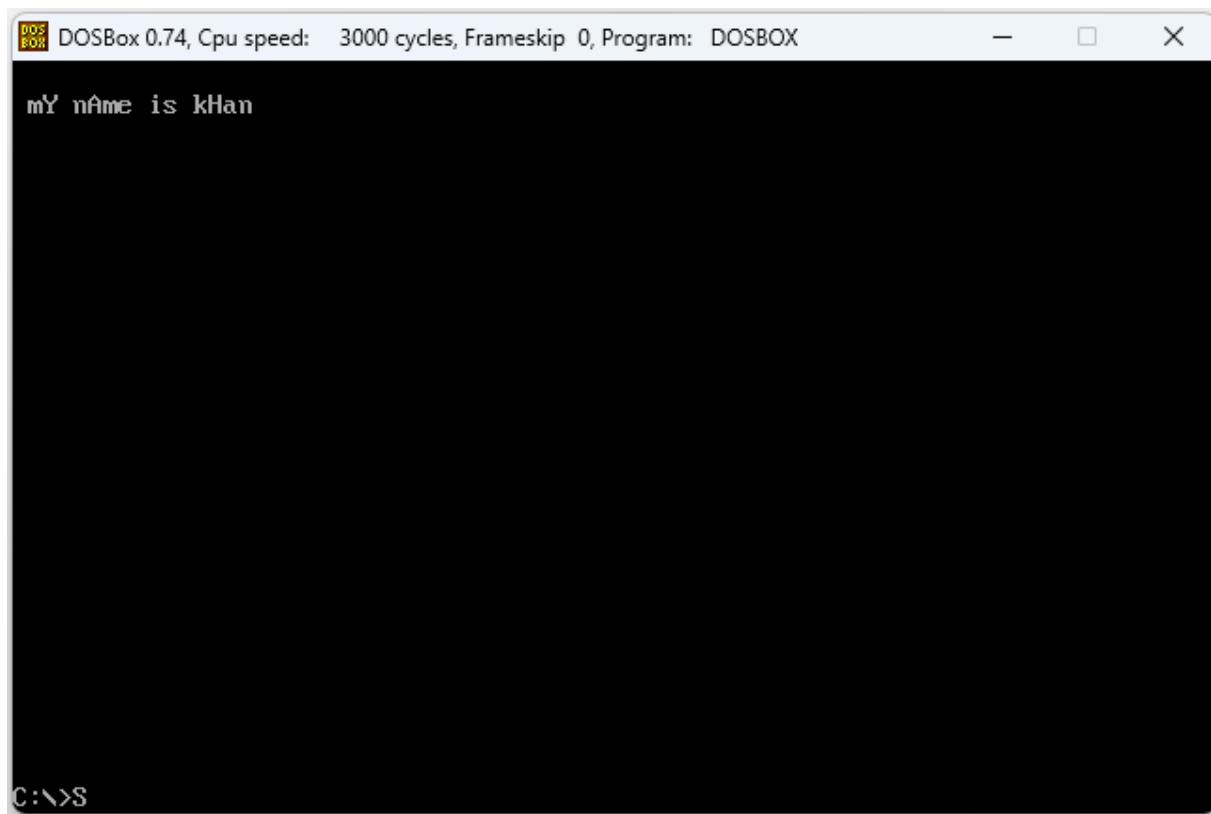
```
push cs
pop es
```

```
mov bp, string
int 0x10
```

////

```
mov ax, string ;(+6)
push ax
mov ax, 15 ;length (+4)
push ax
call convert
```

```
mov ax, 0x4c00
int 0x21
```



Askii of array

2	3	4	5	6	7	8	9	A	B	C	D	E	F	0	1
DS:0103	6D	59	20	6E	41	6D	65	20	69	73	20	6B	48	61	6E

Q2 (ii) Print the triangle as given output, calculate location, clear screen, use subroutine and pass value through stack.

```
org 0x100
```

```
jmp start
```

```
clearscreen:
```

```
    push ax
```

```
    push es
```

```
    push cx
```

```
    push di
```

```
    mov ax, 0xb800
```

```
    mov es, ax
```

```
    mov di, 0
```

```
    mov cx, 2000
```

```
    mov ax, 0x0720
```

```
    rep stosw
```

```
    pop di
```

```
    pop cx
```

```
    pop es
```

```
    pop ax
```

```
    ret
```

```
triangle:
```

```
    push bp
```

```
    mov bp, sp
```

```
    push ax
```

```
    push cx
```

```
    push dx
```

```
    push es
```

```
    push di
```

```
    push si
```

```
    mov ax, 0xb800
```

```
    mov es, ax
```

```
    mov al, 80
```

```
    mul byte[bp+10]
```

```
    add ax, [bp+8]
```

```
    shl ax, 1
```

```
    mov di, ax ;location
```

```
    mov si, ax ;copy location
```

```
    mov cx, [bp+4]
```

```
    add cx, 1
```

```
    mov bx, 1 ; used to know the number of stars to print in on eline
```

```
    mov ax, [bp+6]
```

```
nextline:
```

```
mov di, si
mov dx, 0
```

innerloop:

```
mov [es:di], ax
add di, 2
inc dx
cmp dx, bx
jne innerloop
```

```
add si, 160
inc bx
cmp bx, cx
jl nextline
```

```
pop si
pop di
pop es
pop dx
pop cx
pop ax
pop bp
ret 8
```

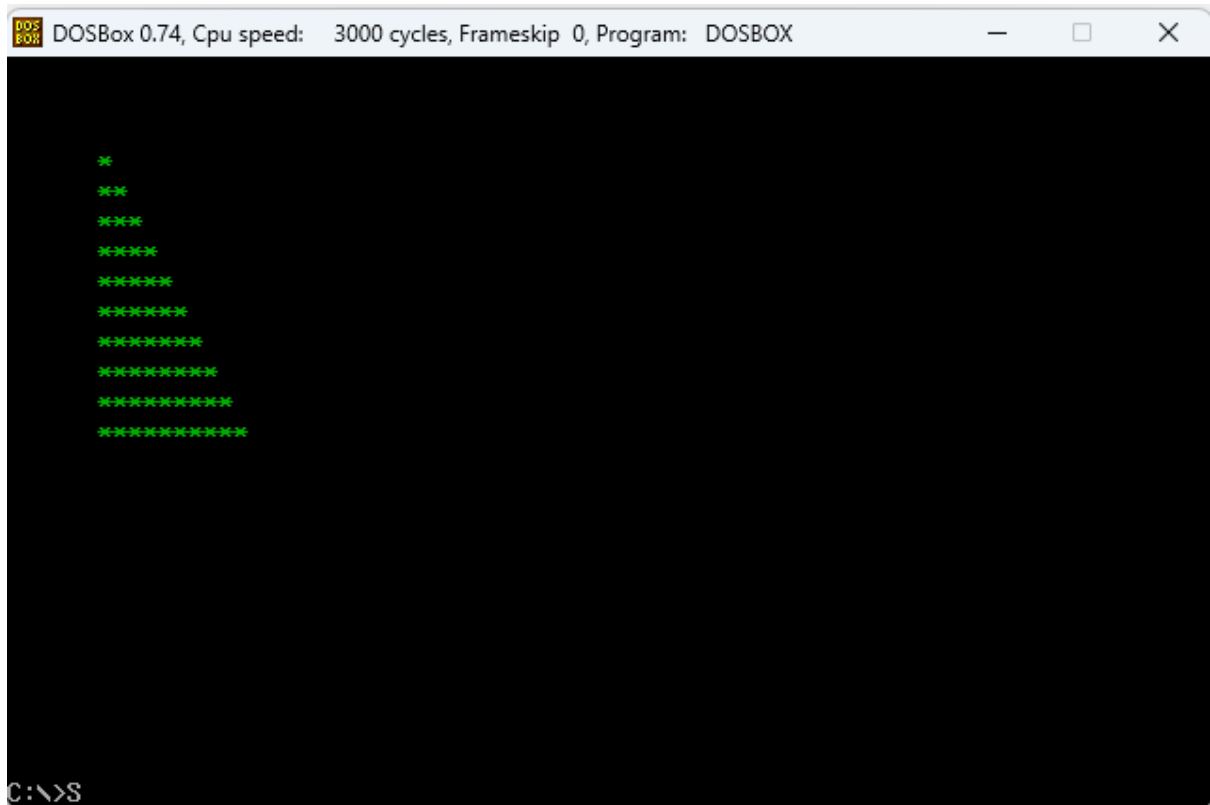
start:

```
call clearscreen
```

```
mov ax, 4      ;y value +10
push ax
mov ax, 6      ;x value +8
push ax
mov ah, 0x02   ;attribute
mov al, 0x2a   ;char
push ax        ;+6
mov ax, 10 ;lenght +4
push ax
```

```
call triangle
```

```
mov ax, 0x4c00
int 0x21
```

Q3 Use hardware interrupt, hook and print the string fast cfd campus when user press tab and when space is pressed print the reversed string, and terminate the program when ESC is pressed. Scan code of Tab (0x0f) and Space (0x39).

```
org 0x100
jmp start
string: db 'fast cfd campus', 0
oldisr: dd 0
clearscreen:
    push ax
    push es
    push cx
    push di

    mov ax, 0xb800
    mov es, ax
    mov di, 0

    mov cx, 2000
    mov ax, 0x0720

    rep stosw

    pop di
    pop cx
    pop es
    pop ax
    ret
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
printstr:
    push bp
    mov bp, sp
    push ax
    push cx
    push es
    push di
    push si

    mov ax, 0xb800
    mov es, ax

    mov al, 80
    mul byte[bp+8]
    add ax, [bp+6]
    shl ax, 1
    mov di, ax      ;location

    mov cx, [bp+12];length
    mov si, [bp+4] ;offset
    mov ax, [bp+10] ;attribute
    mov al, 0

    cld
```

next:

```
lodsb
stosw
loop next
```

```
pop si
pop di
pop es
pop cx
pop ax
pop bp
ret 10
```

////////////////////////////////////

printrevstr:

```
push bp
mov bp, sp
push ax
push cx
push es
push di
push si
```

```
mov ax, 0xb800
mov es, ax
```

```
mov al, 80
mul byte[bp+8]
add ax, [bp+6]
shl ax, 1
mov di, ax      ;location
```

```
mov cx, [bp+12]      ;length
mov si, [bp+4] ;offset
add si, 14 ;points to last of string
mov ax, [bp+10]      ;attribute
mov al, 0
```

next1:

```
std
lodsb
cld
stosw
loop next1
```

```
pop si
pop di
pop es
pop cx
pop ax
pop bp
```

ret 10

;;;;;;;;;

myKBISR:

push ax
push bx

in al, 0x60 ;keystroke

cmp al, 0x0f ;tab
jne nextkeycheck

;;;;;;;;

mov bx, 15 ;length (+12)
push bx
mov bh, 0x02 ;attribute(+10)
push bx
mov bx, 1 ;y value (+8)
push bx
mov bx, 1 ;x value (+6)
push bx
mov bx, string ;(+4)
push bx
call printstr

;;;;;;;;

jmp exit

nextkeycheck:

cmp al, 0x39 ;space
jne nomatch

;;;;;;;;

mov bx, 15 ;length (+12)
push bx
mov bh, 0x02 ;attribute(+10)
push bx
mov bx, 1 ;y value (+8)
push bx
mov bx, 1 ;x value (+6)
push bx
mov bx, string ;(+4)
push bx
call printrevstr

;;;;;;;;

exit:

nomatch:

pop bx

```
pop ax
jmp far [cs:oldisr]
```

```
////////////////////////////////////
```

```
start:
```

```
    call clearscreen
```

```
    mov ax, 0
    mov es, ax
```

```
    mov ax, [es:9*4]
    mov [oldisr], ax
    mov ax, [es:9*4+2]
    mov [oldisr+2], ax
```

```
    cli
    mov word [es:9*4], myKBISR
    mov word [es:9*4+2], cs
    sti
```

```
l1:
```

```
    mov ah, 0
    int 0x16
```

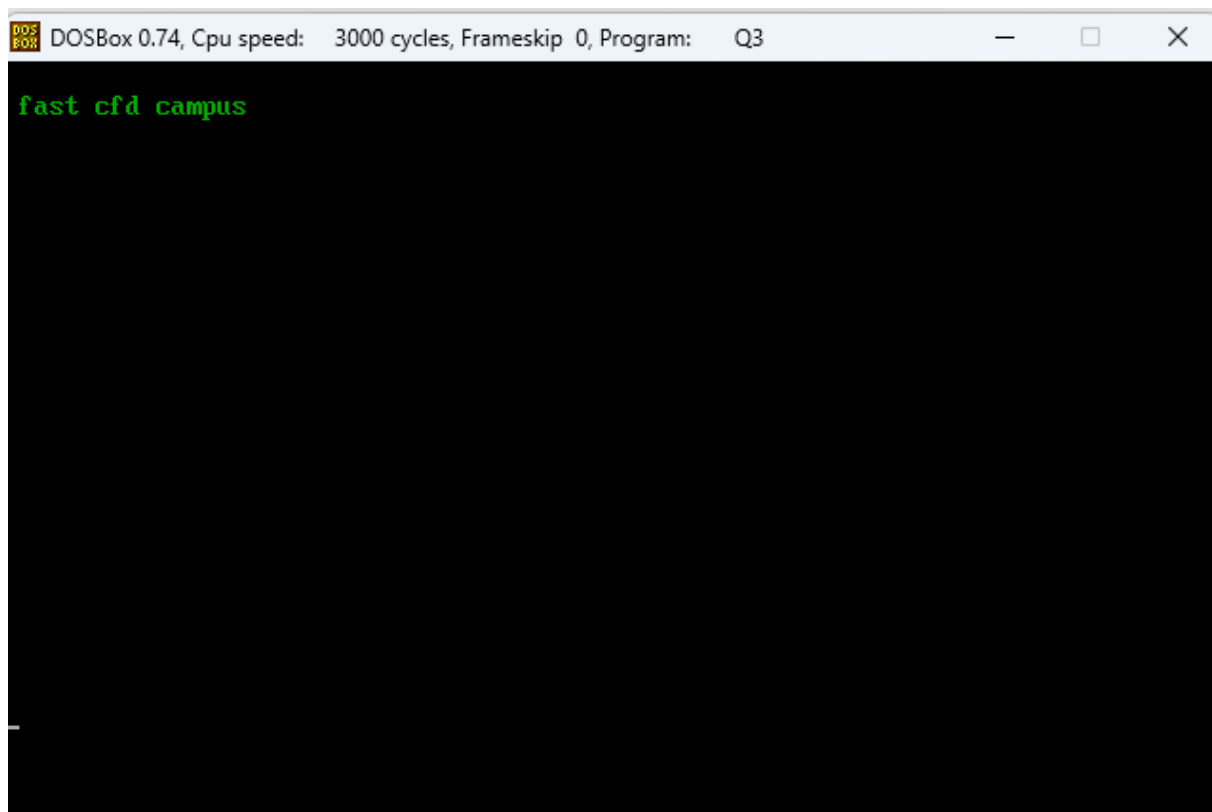
```
    cmp al, 27          ;exit if esc is pressed
    jne l1
```

```
    mov ax, [oldisr]
    mov bx, [oldisr+2]
```

```
    cli
    mov word [es:9*4], ax
    mov word [es:9*4+2], bx
    sti
```

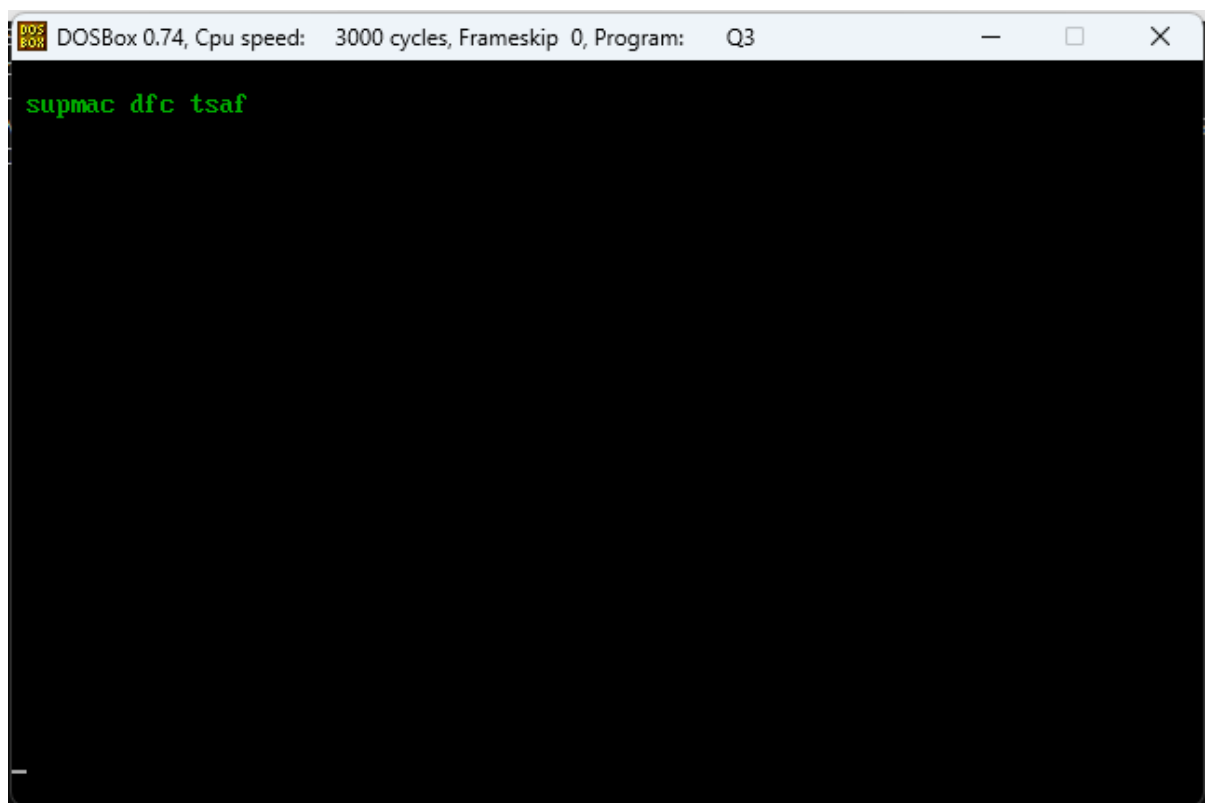
```
    mov ax, 0x4c00
    int 0x21
```

WHEN TAB IS PRESSED PRINTS STRING



A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: Q3". The main window area is black with the text "fast cfd campus" displayed in green monospace font at the top left.

WHEN SPACE IS PRESSED PRINTS REVERSE STRING



A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: Q3". The main window area is black with the text "supmac dfc tsaf" displayed in green monospace font at the top left.